

从0开始写一个AI Agent

赋予代码思考与行动的能力

本次分享大纲



1. 什么是Agent? : 从我们熟悉的大语言模型 (LLM) 出发, 揭示其局限性, 引出Agent的核心定义。



2. ReAct模式: Agent的思考引擎: 深入剖析主流的ReAct运行模式, 理解Agent如何实现“思考-行动”的循环。

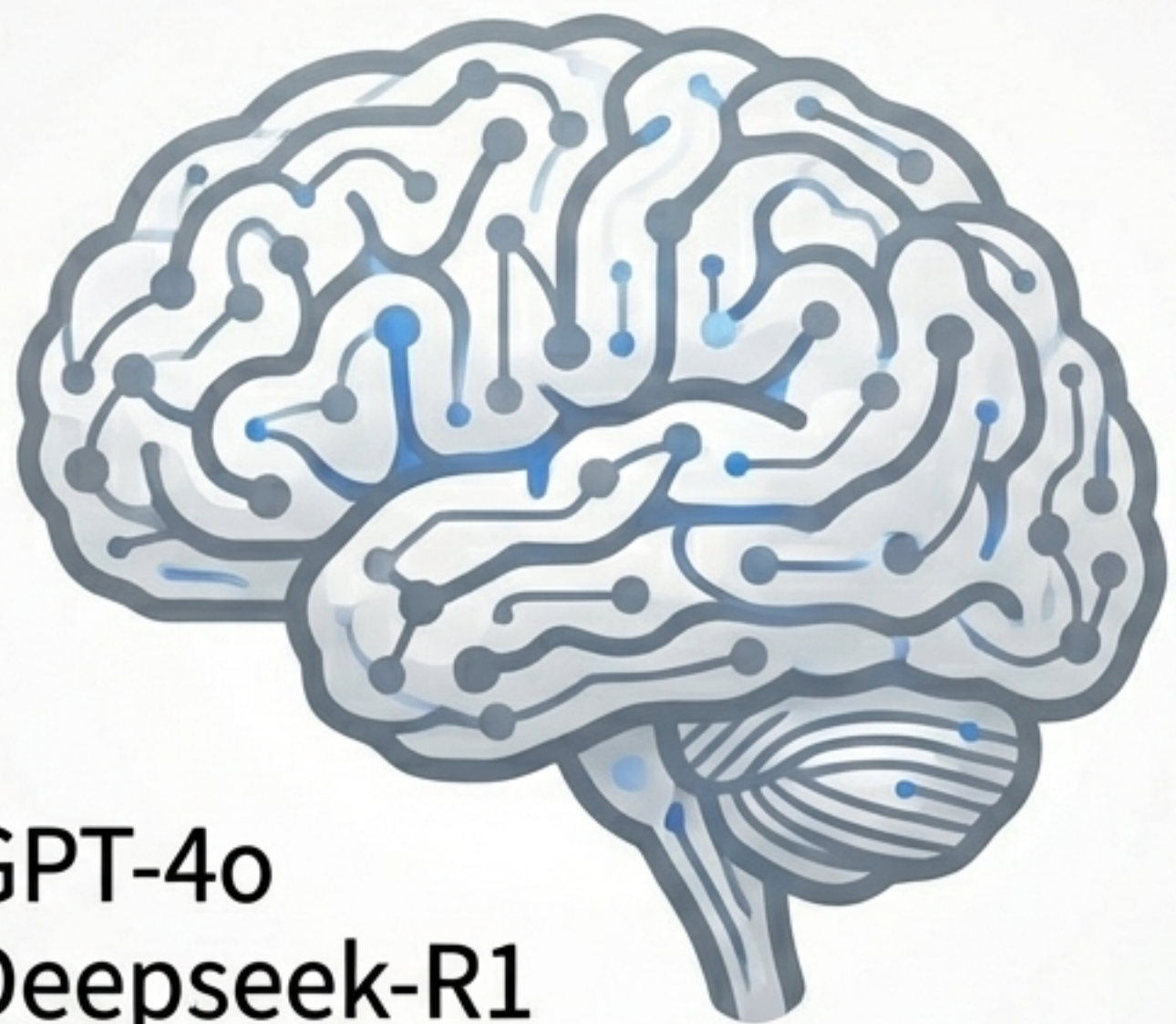


3. 动手实现一个ReAct Agent: 通过具体的Python代码, 一步步构建我们自己的Agent。



4. 举一反三: 探索其他Agent模式: 介绍Plan and Execute模式, 拓宽视野, 并总结核心原理。

一切的起点：强大的大语言模型（LLM）



GPT-4o
Deepseek-R1
Qwen3

我们都用过大模型，比如GPT-4o，Deepseek，Qwen。

- 它们是强大的“知识库”和“回答引擎”。
- **核心能力：**擅长回答各种问题，无论是技术细节还是生活常识。

瓶中之脑：LLM的根本局限

核心问题：无法感知或改变外部环境。



帮我写一个贪吃蛇小游戏。

没问题，代码如下：...



很好，现在帮我给这个游戏加上计分功能。

请把已有的代码复制给我，这样我才能回答。



结论：LLM没有记忆，也无法直接操作文件。它是一个纯粹的文本生成器，与你的开发环境是隔离的。

解决方案：为大脑接上“手”和“脚”



“工具”是什么？

- 任何可以被代码调用的功能，例如：
 - 读写文件内容
 - 查看文件列表
 - 运行终端命令

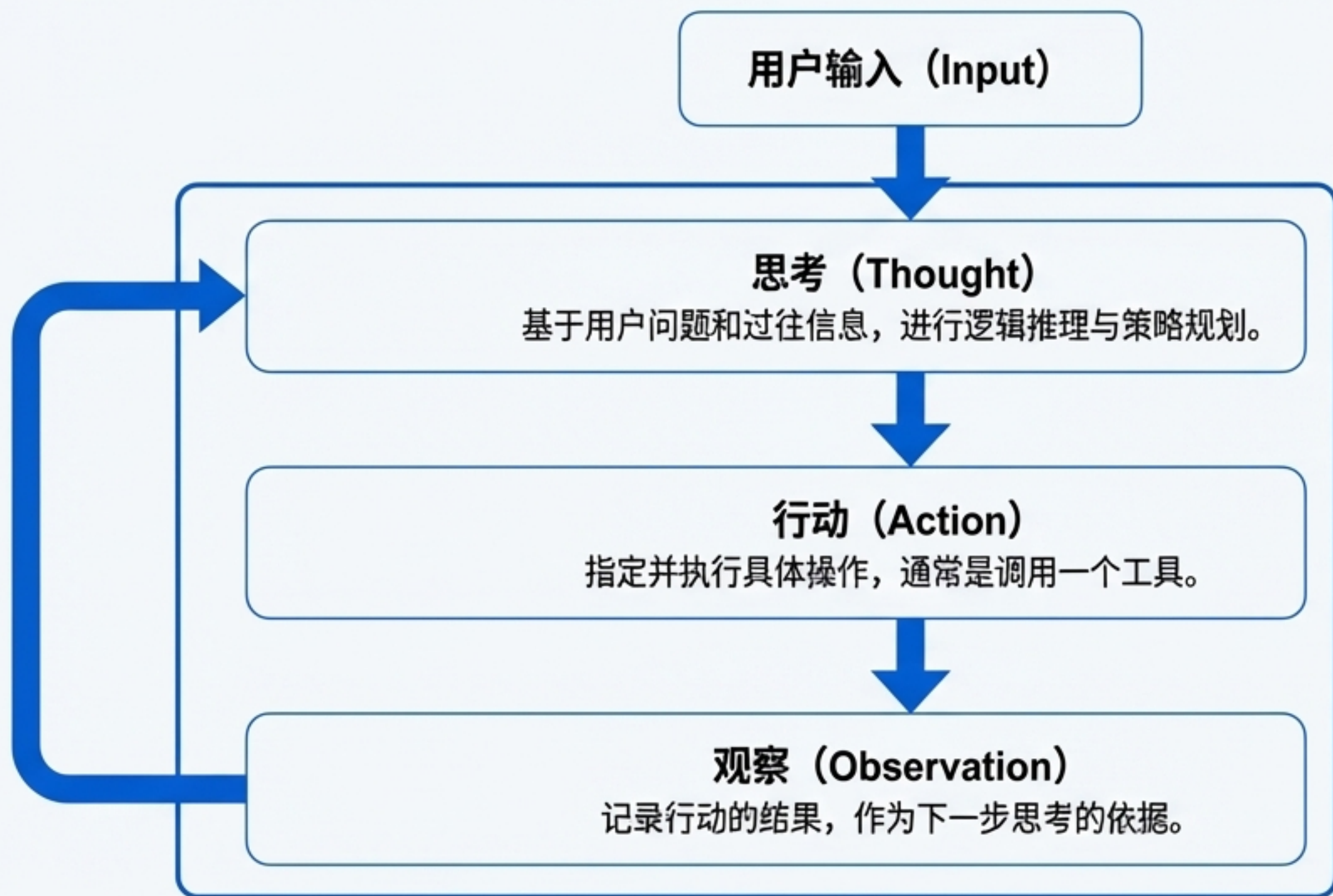
Agent的设计模式：百花齐放，但从核心开始

Agent的设计模式有很多种，每种都有其适用的场景。我们今天将聚焦于最核心、最基础也是应用最广泛的模式：**ReAct**。

设计模式	核心思想	适用场景	典型代表/框架
ReAct	“思考 → 行动 → 观察”动态循环，兼顾环境反馈的循序 渐进任务	动态环境、需实时反馈的任务	ReAct 论文、LangChain 实现
Plan & Execute	先全局规划，再分步复杂、多步骤、目标明确的任务	目标明确的任务	AutoGPT（早期）、BabyAGI
Reflex Agent	无记忆，仅根据当前简单、规则明确、低延迟的输入直 接响应	低延迟聊天机器人、客服问答 等	N/A
Memory Agent	引入记忆机制，支持长对话、个性化、状态依赖任务	长对话（常对带记忆）、 MemGPT	ChatGPT
Multi-Agent	多个Agent协作，分布系统、模拟环境、人工或竞争完成 任务	复杂系统、博弈场景	AutoGPT、CrewAI、MetaGPT
Tool-Calling Agent	以调用外部工具为主需连接API、数据库、执行动作	连接API、数据库、执行动作	OpenAI Function Calling、 LangChain

ReAct模式：Reasoning and Acting（思考与行动）

核心流程：ReAct通过一个动态的“思考-行动-观察”循环来解决复杂问题。



该模式通过动态‘思考-行动-观察’循环，使AI能结合内部推理与外部交互，逐步解决需多步骤处理的复杂任务。

ReAct的实现秘诀：系统提示词（System Prompt）

系统提示词（System Prompt）



本质上，提示词就是Agent的“操作系统”和“行为准则”。

ReAct实战：以“开发贪吃蛇游戏”为例

Question

“帮我写一个贪吃蛇游戏，需要html，css，js三个文件”

第一轮循环

Thought

我需要创建三个文件：HTML、CSS和JavaScript。HTML文件是骨架，我将从它开始。

Action

```
write_to_file('snake.html', '<!DOCTYPE html>...')
```

Observation

写入成功

第二轮循环

Thought

HTML文件已创建成功。现在我需要为游戏创建样式。

Action

```
write_to_file('snake.css', '#game-board { ... }')
```

Observation

写入成功

...以此类推，直到任务完成。

动手编码：一个ReAct Agent的三个核心组件

我们将通过Python代码实现Agent。其结构可以分为三个部分：



Agent之魂：解读 `prompt\_template.py`

这个模板告诉LLM如何思考、如何行动、以及如何格式化它的输出。

规则定义

强制模型输出必须包含`<thought>`、`<action>`或`<final\_answer>`等XML标签。

```
react_system_prompt_template = """
...
所有步骤的回答都必须包含以下 XML 标签之一: <thought>, <action> 或
<final_answer>。
...
```

工具描述

清晰列出每个可用工具的名称和功能说明 (e.g., `read\_file(file\_path):` 用于读取文件内容)。

```
...
本次任务可用工具:
${tool_list}
```

环境信息

注入动态信息，如当前操作系统和文件列表，让Agent感知环境。

```
环境信息:
操作系统: ${operating_system}
...
"""
```


Agent之肢：实现工具函数

工具就是普通的Python函数，Agent通过解析LLM的输出并调用它们来与外部世界交互。

```
# agent.py

def read_file(file_path):
    """用于读取文件内容"""
    with open(file_path, "r", encoding="utf-8") as f:
        return f.read()

def write_to_file(file_path, content):
    """用于将内容写入指定文件"""
    with open(file_path, "w", encoding="utf-8") as f:
        f.write(content.replace("\n", "\n"))
    return "写入成功"

def run_terminal_command(command):
    """用于执行终端命令"""
    run_result = subprocess.run(command, shell=True, capture_output=True, text=True)
    # ... (error handling)
```


Agent之核：`agent.py` 中的 `run` 循环（上）

这是驱动Agent的核心逻辑，一个无限循环，直到找到最终答案。

```
1 # class ReactAgent:
2   def run(self, user_input: str):
3       # ... (初始化 messages)
4
5       while True:
6           # 1. 调用大模型
7           content = self.call_model(messages)
8
9           # 2. 提取 Thought
10          thought_match = re.search(r'<thought>(.*?)</thought>', content, re.DOTALL)
11
12          # 3. 检查 Final Answer
13          if "<final_answer>" in content:
14              final_answer = re.search(r'<final_answer>(.*?)</final_answer>', ...).group(1)
15              return final_answer
16
17          # 4. 提取 Action
18          action_match = re.search(r'<action>(.*?)</action>', content, re.DOTALL)
19          # ...
```

1. 调用大模型：发送当前上下文（消息历史）给 LLM，获取它的响应内容。

2. 提取 Thought：从 LLM 的响应中解析出思考过程（<thought>标签内的内容）。

3. 检查 Final Answer：判断 LLM 是否已经提供了最终答案（<final_answer>标签）。如果是，则提取并返回。

4. 提取 Action：如果还没有最终答案，则解析出需要执行的动作（<action>标签内的内容）。

Agent之核：`agent.py` 中的 `run` 循环（下）

在提取思考和行动后，Agent执行工具并将结果反馈给下一轮循环。

```
20 # ... (接上页)
21     action = action_match.group(1)
22
23     # 5. 解析工具名称和参数
24     tool_name, args = self.parse_action(action)
25
26     # 6. 执行工具
27     try:
28         observation = self.tools[tool_name](*args)
29     except Exception as e:
30         observation = f"执行出错: {e}"
31
32     # 7. 将观察结果加入历史记录
33     obs_msg = f"<observation>{observation}</observation>"
34     messages.append({"role": "user", "content": obs_msg})
```

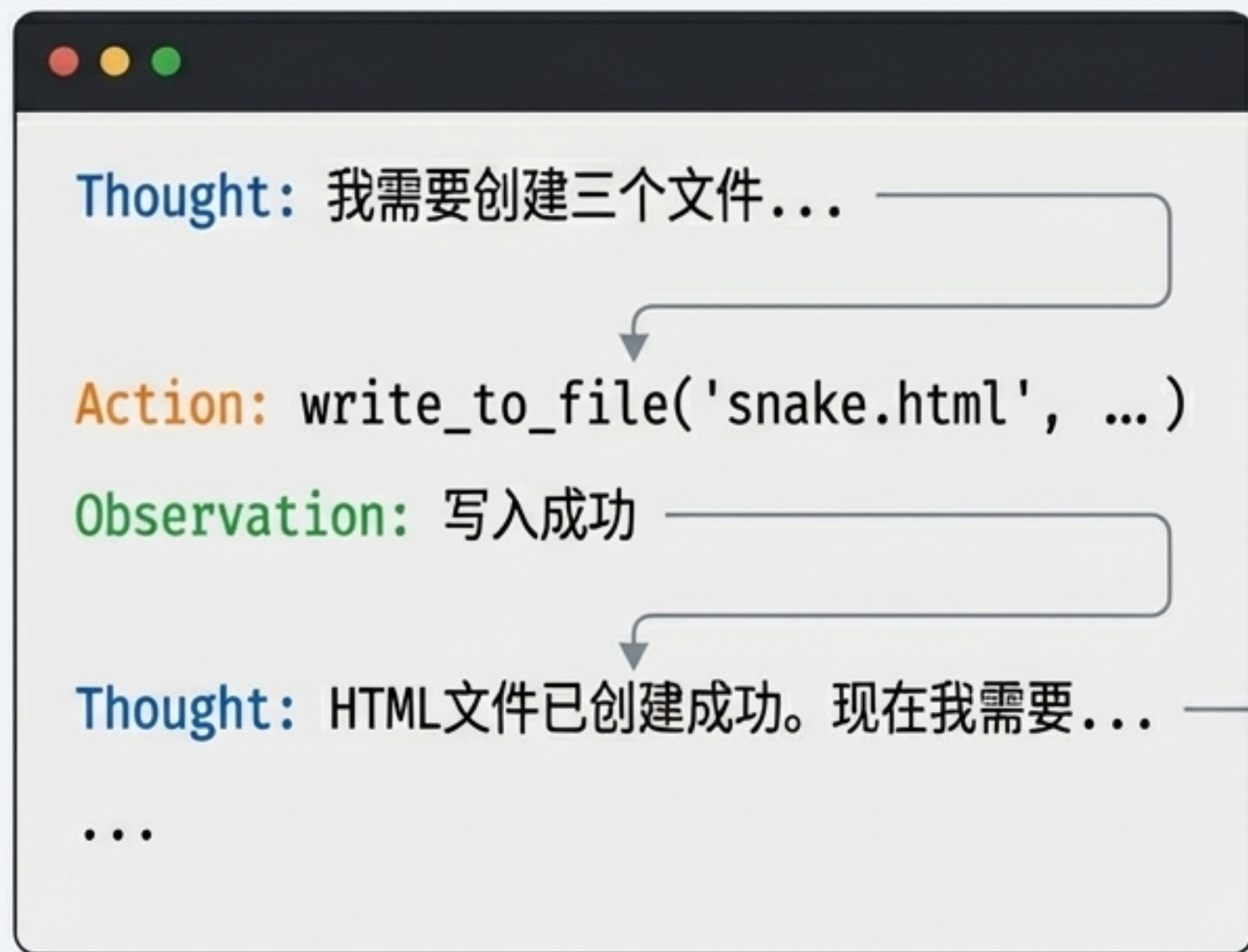
5. 解析工具名称和参数：从 <action> 标签中提取工具名称和参数列表。

6. 执行工具：调用对应的 Python 工具函数。包含错误处理，确保执行失败时也能继续。

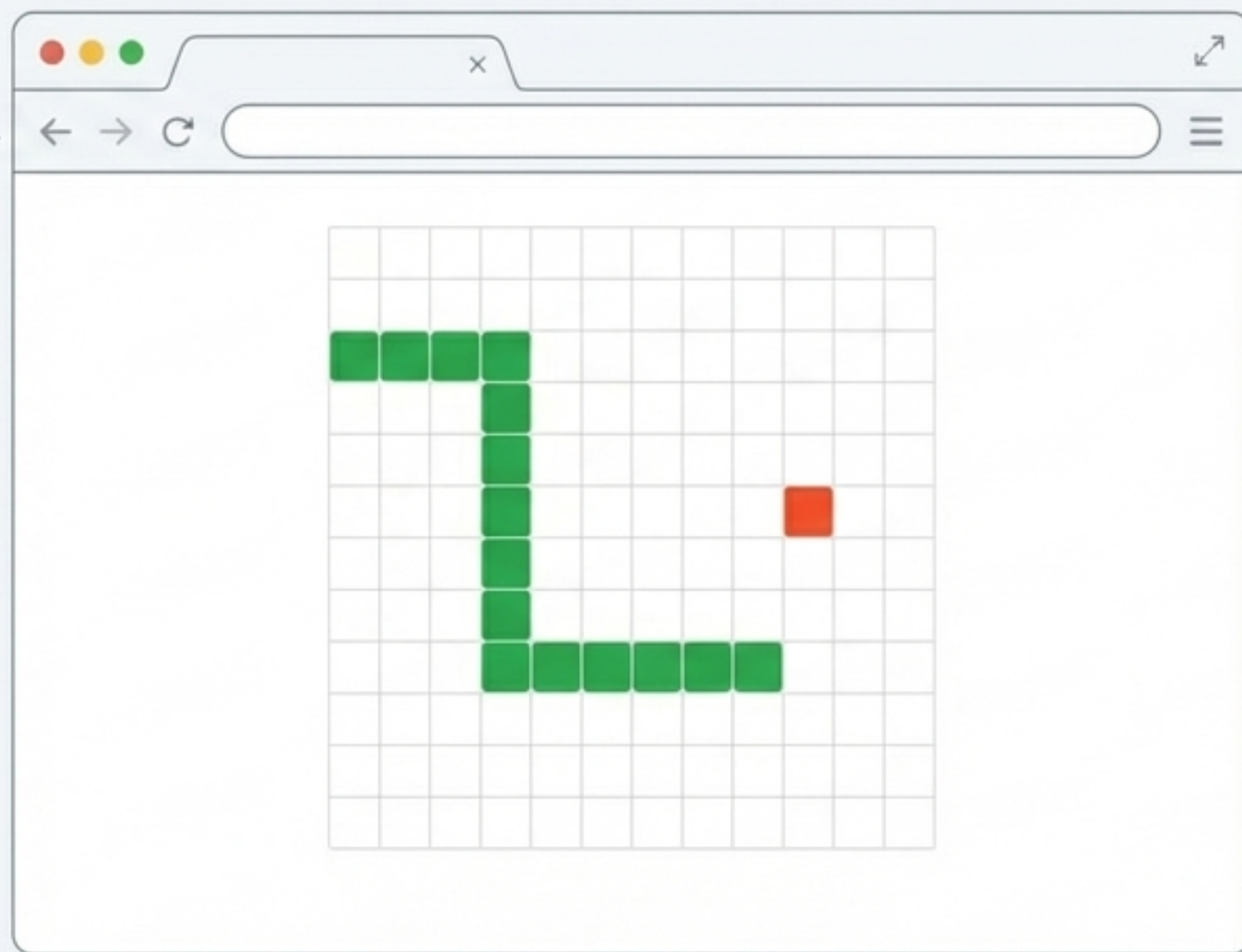
7. 将观察结果加入历史记录：将工具执行结果包装在 <observation> 标签中，作为 用户，作为新的用户消息加入对话历史，供下一轮思考使用。

见证奇迹：Agent自主完成任务

Agent的思考过程（终端输出）



最终成果（浏览器）



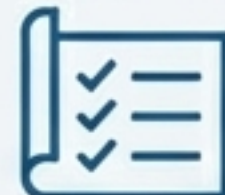
拓宽思路：对比ReAct的Plan and Execute模式

ReAct（战术型）



走一步，看一步。每一步都基于前一步的观察结果进行动态调整。

Plan and Execute（战略型）



先规划，后执行。首先生成一个完整的、多步骤的计划，然后逐一执行。

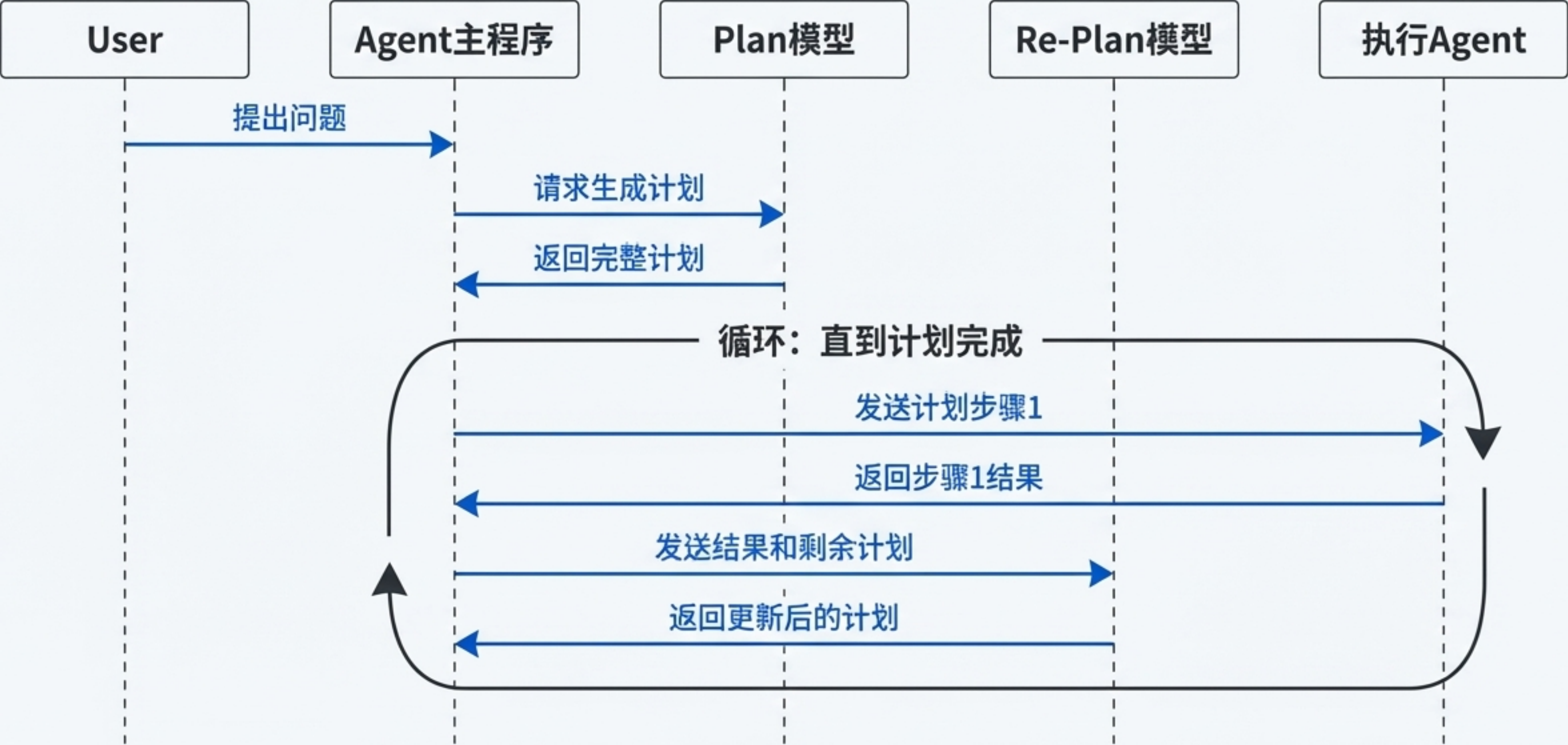
适用场景 & 示例：适用于目标明确、步骤可以提前规划的复杂任务。

问题：“现任美国总统的家乡在哪？”

Plan:

1. 查询当前时间。
2. 查询对应日期的美国总统名字。
3. 查询这位总统的家乡。

Plan and Execute 工作流程解析



核心原理总结



1. Agent = 大模型（大脑）+ 工具（行动）

Agent的本质是赋予了LLM与外部世界交互、执行任务的能力。



2. ReAct = 思考 → 行动 → 观察

这是实现自主问题解决的基础循环，让Agent能够根据环境反馈动态调整策略。



3. 提示词即代码（Prompt is the Code）

Agent的复杂行为是由精心设计的System Prompt引导的，而不是复杂的硬编码逻辑。掌握Prompt就是掌握了定义Agent行为的关键。

你已经掌握了构建AI Agent的基础

真正的创造力不仅在于编写Agent的循环逻辑，更在于如何创造性地定义**工具**和设计**思考过程**。

你会赋予你的代码去解决什么问题？